

UNIVERSIDADE FEDERAL DE ITAJUBÁ

Clara Leal Moreno - 2022003835
João Luiz de Miranda Cilli - 2022003942
João Pedro Basílio Dinareli - 2022002892
Júlia Furtado Araújo - 2023005469

DETECTOR DE CHOROS DE BEBÊS TINYML - IESTI01

Itajubá
2025

1 INTRODUÇÃO

A principal forma de comunicação de um bebê antes do desenvolvimento da fala é o choro, utilizado para expressar necessidades e desconfortos diversos, como fome, dor ou medo. Esta etapa da infância demanda atenção constante dos pais ou responsáveis, que precisam estar vigilantes para garantir o bem-estar da criança. No entanto, o monitoramento contínuo pode ser desafiador, especialmente em ambientes com ruído ou quando o cuidador está em outro cômodo.

2 OBJETIVO

Este projeto propõe utilizar os conhecimentos de Inteligência Artificial (IA) em sistemas embarcados, conhecida como Tiny Machine Learning (TinyML), para o desenvolvimento de um sistema capaz de identificar padrões acústicos de choro de bebê em tempo real, para posteriormente alertar os responsáveis de forma eficaz. Nesta primeira versão, optou-se pelo uso de LEDs como forma de alerta.

3 HARDWARE UTILIZADO

Para a execução desse projeto foram utilizados os seguintes componentes:

- Placa Microcontroladora: Arduino Nano 33 BLE Sense

Processador: ARM Cortex-M4 de 32 bits rodando a 64 MHz.

Memória: 1 MB de Flash e 256 KB de SRAM.

- Sensor de Áudio: Microfone Digital MP34DT05-A

Este microfone omnidirecional já vem integrado à placa Arduino Nano 33 BLE Sense, sendo o principal componente para a aquisição dos dados acústicos do ambiente.

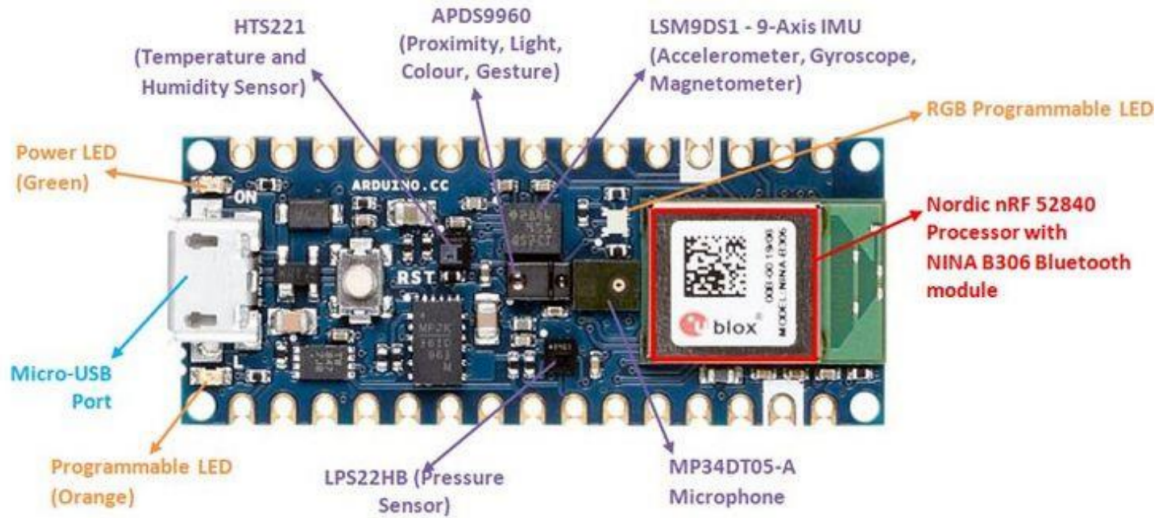


Figura 1: Detalhes da placa Arduino Nano 33 BLE Sense.

4 CONSTRUÇÃO E PRÉ-PROCESSAMENTO DO DATASET

Para o treinamento e teste do modelo de classificação, foi construído um dataset customizado a partir da combinação de duas fontes de dados públicas, visando abranger duas classes principais: `cry` (choro de bebê) e `not_cry` (outros sons, incluindo silêncio e ruído ambiente).

4.1 Fontes de Dados

O conjunto de dados foi composto a partir das seguintes fontes:

1. **Infant Cry Dataset (SADHISH, 2022):** Esta foi a fonte primária para ambas as classes. O dataset original contém amostras de áudio já categorizadas, sendo utilizadas as amostras de choro de bebê para a classe `cry` e áudios de outros sons (como fala, ruído de fundo) para a classe `not_cry`.
2. **QMSAT Dataset (KHAN, 2021):** Durante os testes preliminares, observou-se que o modelo poderia apresentar dificuldade em distinguir o choro de períodos de silêncio ou ruído ambiente de baixa intensidade. Para mitigar este problema e reforçar a classe `not_cry`, foram incorporadas amostras da classe `Normal(Silence)`.

deste segundo dataset.

4.2 Pré-processamento e Padronização

Todos os arquivos de áudio foram submetidos a um processo de padronização para garantir a consistência dos dados de entrada do modelo. As etapas foram:

- **Frequência de Amostragem:** Todos os áudios foram normalizados para uma frequência de amostragem de **16 kHz**. Esta taxa é suficiente para capturar as características vocais do choro de bebê, mantendo o tamanho dos arquivos gerenciável.
- **Janelamento (Windowing):** Os áudios contínuos foram segmentados em janelas fixas de **1 segundo**. Essa duração foi definida como um balanço ideal para capturar um evento de choro completo e, ao mesmo tempo, ser compatível com os requisitos de processamento em tempo real do microcontrolador.

Após o pré-processamento, o dataset final totalizou **789 amostras de áudio** independentes.

4.3 Estrutura Final e Divisão do Dataset

O conjunto de dados final foi dividido na proporção de 80% para treinamento e 20% para teste, garantindo que não houvesse sobreposição de amostras entre os dois conjuntos. A distribuição das classes está detalhada na Tabela 1.

Tabela 1: Distribuição das amostras do dataset.

Classe	Treinamento (80%)	Teste (20%)	Total
cry	420	105	525
not_cry	210	54	264
Total	630	159	789

Nota-se um desbalanceamento deliberado em favor da classe **cry**, uma estratégia adotada para garantir que o modelo tenha exposição suficiente à classe de maior interesse (o evento a ser detectado), o que é comum em problemas de detecção de anomalias.

5 PRÉ-PROCESSAMENTO

Os modelos foram criados utilizando três blocos distintos de pré-processamento: Espectrograma, *Mel-Filterbank Energy* (MFE) e *Mel-Frequency Cepstral Coefficients* (MFCC), com o objetivo de analisar a contribuição de cada um para a detecção de choros de bebês.

5.1 Espectrograma

O bloco de Espectrograma processa o sinal para obter informações de tempo e frequência. Destaca-se na análise de dados de áudio fora do contexto de reconhecimento de voz, assim como em dados de sensores com frequências contínuas.

Este bloco extrai características de tempo e frequência ao dividir o sinal em quadros, aplicar *Fast Fourier Transform* (FFT) em cada um e gerar um mapa de intensidades espectrais ao longo do tempo.

Os parâmetros utilizados neste bloco estão descritos na Tabela 2.

Tabela 2: Parâmetros do Espectrograma

Parâmetro	Valor
Frame length	0.004
Frame stride	0.004
FFT size	32
Noise floor (dB)	-81

5.2 MFE

De forma similar ao bloco de Espectrograma, o bloco de pré-processamento MFE também extrai informações de tempo e frequência do sinal, porém, utiliza uma escala não linear de frequência, a *Mel-Scale*. Assim, esse método é eficaz na análise de dados de áudio em contextos que não envolvem reconhecimento de voz, especialmente quando os sons são perceptíveis ao ouvido humano.

O bloco MFE funciona de forma semelhante ao Espectrograma, utilizando os mesmos parâmetros de quadro e FFT. A diferença é que, após calcular o Espectrograma, ele aplica

filtros triangulares na *Mel-Scale* para extrair bandas de frequência perceptíveis ao ouvido humano, com maior concentração de filtros nas baixas frequências e menor nas altas.

Os parâmetros ajustáveis para este bloco são chamados de *Mel-filterbanks energy features* e os que foram utilizados estão descritos na Tabela 3.

Tabela 3: Parâmetros do MFE

Parâmetro	Valor
Frame length	0.025
Frame stride	0.01
Filter number	41
FFT length	512
Low frequency	80
High frequency	-
Noise floor (dB)	-100

5.3 MFCC

De forma similar ao bloco MFE, o bloco de pré-processamento MFCC extrai a energia das bandas usando a *Mel-Scale* e acrescenta um passo extra: aplica-se uma *Discrete Cosine Transform* (DCT) nos filtros para obter coeficientes cepstrais que representam melhor as características acústicas do áudio. Por isso, é o padrão para reconhecimento de voz, mas também pode ser eficaz em alguns casos que não envolvem voz humana.

Os parâmetros utilizados neste bloco estão descritos na Tabela 4.

Tabela 4: Parâmetros do MFCC

Parâmetro	Valor
Number of coefficients	13
Frame length	0.025
Frame stride	0.02
Filter number	32
FFT length	512
Normalization window size	151
Low frequency	80
High frequency	-
Coefficient	0.98

6 ARQUITETURA DA REDE NEURAL

A estrutura sequencial do modelo é composta pelas seguintes camadas principais, detalhadas abaixo e ilustradas na Figura 2:

- **Camada de Entrada (*Input Layer*):** Recebe o vetor de características (features) extraído na etapa de pré-processamento. Sua dimensão de entrada varia conforme o bloco de processamento de sinal utilizado (i.e. MFCC, MFE, Espectrograma).
- **Blocos Convolucionais (1D Conv/Pool):** O modelo utiliza dois blocos convolucionais para extrair padrões hierárquicos dos dados. Cada bloco é composto por:
 - Uma camada convolucional 1D (**Conv1D**) para atuar como um detector de características. O primeiro bloco possui 8 filtros e o segundo, 16. Ambos usam um *kernel* de tamanho 3.
 - Uma camada de *Max Pooling* para reduzir a dimensionalidade dos dados e focar nas características mais proeminentes.
- **Camadas de Regularização (*Dropout*):** Após cada bloco convolucional, uma camada de *Dropout* com taxa de 0.25 é aplicada. Sua função é desativar aleatoriamente 25% dos neurônios durante o treinamento para prevenir o superajuste (*overfitting*) e melhorar a generalização do modelo.

- **Camada de Achatamento (*Flatten Layer*):** Transforma a matriz de características resultante dos blocos convolucionais em um vetor unidimensional, preparando os dados para a camada de saída.
- **Camada de Saída (*Output Layer*):** Uma camada densa com 2 neurônios, correspondentes às classes `cry` e `not_cry`. Utiliza a função de ativação *softmax* para converter os resultados da rede em uma distribuição de probabilidades, indicando a confiança da previsão para cada classe.

As dimensões exatas das camadas de entrada e de *Reshape* (não detalhada na lista, mas presente no modelo) são ajustadas automaticamente pela plataforma *Edge Impulse* para garantir a compatibilidade entre o bloco de pré-processamento e a entrada da rede neural. A Figura 2 apresenta um diagrama visual detalhado de cada arquitetura testada.



Figura 2: Arquiteturas para cada bloco de pré-processamento.

7 TREINAMENTO E DEPLOYMENT

O desenvolvimento do sistema de detecção de choro seguiu um fluxo de trabalho estruturado, abrangendo as etapas de treinamento, teste e *deployment* do modelo. Todas estas etapas foram gerenciadas e executadas com o auxílio da plataforma de desenvolvimento *Edge Impulse*, que provê um ambiente integrado desde a aquisição de dados até a geração da biblioteca para o hardware alvo.

7.1 Treinamento do Modelo

Para cada um dos blocos de pré-processamento definidos na Seção 4 (MFE, MFCC e Espectrograma), um modelo, com a arquitetura descrita na Seção 6, foi treinado de forma independente. O treinamento foi configurado com os seguintes hiperparâmetros principais:

- **Número de Épocas de Treinamento:** O treinamento foi executado por [30] épocas, com exceção do modelo que utiliza Espectrograma, que conta com [60] épocas de treinamento.
- **Taxa de Aprendizagem (*Learning Rate*):** Foi utilizada uma taxa de aprendizagem de [0.005].

7.2 Teste

A performance de cada modelo treinado foi rigorosamente avaliada utilizando o conjunto de teste. As principais métricas de avaliação foram:

- **Acurácia:** Percentual de classificações corretas sobre o total de amostras de teste.
- **Matriz de Confusão:** Uma tabela que visualiza o desempenho do classificador, detalhando os acertos e erros para cada classe (Verdadeiros Positivos, Falsos Positivos, Verdadeiros Negativos e Falsos Negativos). Esta métrica é especialmente importante para identificar se o modelo está falhando mais em uma classe específica, como nos casos de falsos negativos para a classe `cry`.



Figura 3: Métricas do modelo final (MFE 2D) no conjunto de teste.

7.3 Otimização e Deployment

Após a seleção do modelo com melhor desempenho prático (MFE 2D), a etapa final consistiu na sua otimização e implantação no microcontrolador Arduino Nano 33 BLE Sense.

1. **Quantização Pós-Treinamento:** Para tornar o modelo compatível com as restrições de memória do hardware, ele foi submetido a um processo de quantização. Este processo reduz a precisão numérica dos pesos do modelo (convertendo-os de ponto flutuante de 32 bits para inteiros de 8 bits), o que diminui drasticamente o tamanho do arquivo do modelo (de **60.8 KB** para **38.7 KB**) e o consumo de RAM (de **159.4 KB** para **41.8 KB**), além de acelerar o tempo de inferência, geralmente com um impacto mínimo na acurácia.
2. **Geração da Biblioteca Arduino:** A plataforma *Edge Impulse* compilou o modelo quantizado e todos os blocos de pré-processamento em uma biblioteca Arduino.
3. **Integração e Deployment do Código:** A biblioteca gerada foi integrada a um

sketch (código-fonte) customizado na IDE do Arduino. Este *sketch* final é responsável por orquestrar todo o pipeline em tempo real:

- Coletar continuamente amostras de áudio através do microfone PDM integrado.
- Passar os dados de áudio para a função de pré-processamento da biblioteca.
- Executar a inferência do modelo com os dados processados.
- Ler o resultado da classificação (probabilidades para cada classe).
- Acionar a lógica de controle do LED RGB com base no resultado da inferência.

Este processo resultou no protótipo funcional descrito e avaliado na seção de Resultados Práticos (Seção 8).

8 RESULTADOS PRÁTICOS

Após o treinamento e teste, todos os modelos foram submetidos à implantação (*deployment*) na placa Arduino Nano 33 BLE Sense para avaliação de desempenho em um cenário prático. Foi implementada uma interface de resposta visual simples: um LED RGB integrado à placa acende na cor **verde** quando nenhum choro é detectado e na cor **vermelha** ao classificar um áudio como choro.

8.1 Análise Qualitativa do Desempenho Prático

A avaliação prática consistiu na apresentação de amostras de áudio que não estavam presentes no conjunto de dados de treinamento, incluindo diferentes tipos de choro, fala humana e ruídos ambientes.

- **MFE (2D):** Este modelo apresentou o melhor balanço entre as métricas quantitativas e o desempenho prático. A quantidade de falsos negativos foi notavelmente baixa e o tempo de resposta foi satisfatório, sendo, portanto, **considerado o modelo final do protótipo**.
- **MFE (1D) e MFCC (1D):** Ambos os modelos, embora com boa acurácia teórica, demonstraram uma performance prática inferior. Apresentaram uma latência de

inferência perceptível ao usuário (a "demora" para determinar o resultado) e uma maior incidência de falsos negativos em comparação com a versão MFE 2D.

- **Espectrograma (1D):** O modelo baseado em Espectrograma demonstrou-se inadequado para a aplicação prática. Conforme observado nos testes, o modelo sofreu de superajuste (*overfitting*) severo, classificando virtualmente todas as entradas de áudio como "choro", geralmente com uma confiança em torno de 75%, tornando-o inútil para a detecção real.

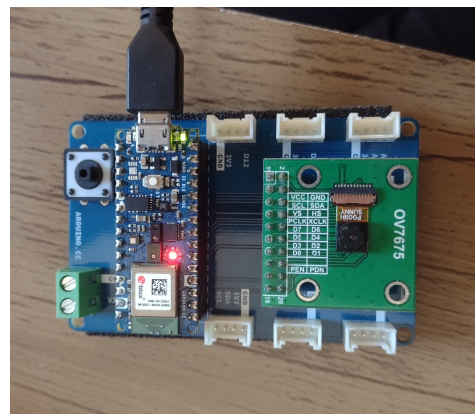
8.2 Demonstração do Protótipo e Recursos do Projeto

O comportamento do protótipo final (utilizando o modelo MFE 2D) é demonstrado na Figura 4. O projeto público no *Edge Impulse* e os códigos-fonte utilizados nos testes estão disponíveis online para consulta.

- **Projeto no Edge Impulse:** <https://studio.edgeimpulse.com/public/726280/live>
- **Códigos-fonte:** Link para o Drive



(a) LED verde: Nenhuma detecção.



(b) LED vermelho: Choro detectado.

Figura 4: Demonstração do protótipo em operação com o modelo MFE 2D.

9 CONCLUSÃO

Este trabalho propôs e desenvolveu um sistema embarcado para a detecção de choro de bebê em tempo real, aplicando conceitos de Tiny Machine Learning. A execução do projeto permitiu extrair conclusões significativas tanto sobre a modelagem do problema quanto sobre os desafios inerentes à implementação em hardware de recursos limitados.

O principal achado técnico foi a constatação de que a escolha do bloco de pré-processamento de áudio é um fator determinante para o desempenho do sistema. Os modelos que utilizaram extratores de características otimizados para a percepção da voz humana, como o MFE (Mel-Frequency Energy) e o MFCC (Mel-Frequency Cepstral Coefficients), apresentaram resultados robustos e viáveis para a aplicação prática. Em contrapartida, o modelo baseado em Espectrograma demonstrou-se ineficaz, falhando em generalizar para novos dados e reforçando a importância da engenharia de características adequadas à natureza específica do sinal de áudio.

Outro aspecto central foi a negociação com as limitações computacionais do microcontrolador Arduino Nano 33 BLE Sense. A restrita disponibilidade de memória Flash e RAM exigiu a otimização contínua da arquitetura da rede neural, forçando um balanço criterioso entre a complexidade do modelo — e sua acurácia potencial — e a sua viabilidade de implantação. Este desafio reitera o pilar fundamental do TinyML: o desenvolvimento de soluções de inteligência artificial que sejam não apenas eficazes, mas também eficientes o suficiente para operar em ecossistemas de borda.

Conclui-se, portanto, que o objetivo principal do projeto foi atingido com sucesso. O protótipo funcional validou a hipótese de que é tecnicamente viável criar sistemas embarcados autônomos para o monitoramento acústico de bebês, oferecendo uma ferramenta de auxílio de baixo custo e consumo energético. O sucesso desta aplicação em um problema do cotidiano demonstra a notável versatilidade e o potencial da tecnologia TinyML para gerar impacto em diversas áreas do bem-estar e cuidado humano.

REFERÊNCIAS BIBLIOGRÁFICAS

KHAN, C. R. Qmsat dataset. Disponível em: <https://www.kaggle.com/datasets/>

crdkhan/qmsat-dataset. 2021.

SADHISH, S. Infant cry dataset. Disponível em: <https://www.kaggle.com/datasetssanmithasadhish/infant-cry-dataset>. 2022.